Η Γλώσσα Προγραμματισμού C++

- ΒΑΣΙΚΕΣ ΔΟΜΕΣ ΠΡΟΓΡΑΜΜΑΤΟΣ (ΑΚΟΛΟΥΘΙΑΚΗ, ΔΙΑΚΛΑΔΩΣΗ, ΕΠΑΝΑΛΗΨΗ))
- ΔΗΜΙΟΥΡΓΙΑ ΣΥΝΑΡΤΗΣΕΩΝ ΑΠΟ ΤΟΝ ΧΡΗΣΤΗ
- ΠΙΝΑΚΕΣ (ΜΟΝΟΔΙΑΣΤΑΤΟΙ – ΔΙΣΔΙΑΣΤΑΤΟΙ)
- ΣΥΜΒΟΛΟΣΕΙΡΕΣ (STRINGS)
- ΑΡΧΕΙΑ (FILES)
- ΑΦΗΡΗΜΕΝΕΣ ΔΟΜΕΣ ΔΕΔΟΜΕΝΩΝ (STRUCTS)

Η Γλώσσα Προγραμματισμού C++

Δημιουργήθηκε το 1979 από τον Bjarne Stroustrup

- Στην αρχή ονομαζόταν C with classes
- Από τις πιο διαδεδομένες και ισχυρές γλώσσες προγραμματισμού
- Περιλαμβάνει όλες τις εντολές της C
- Σημαντικές εφαρμογές που γράφτηκαν σε C++
  - MicrosoftWindows
  - MicrosoftOffice
  - Photoshop
  - Mozilla Firefox
  - MySQL ο μTorrent

- Το IDE περιβάλλον που θα χρησιμοποιήσετε για να γράφετε/μεταγλωττίζετε τα προγράμματα στην τάξη είναι το Code::Blocks
  - δωρεάν IDE (Integrated Development Environment) σχεδιασμένο για τη C/C++ (και όχι μόνο).
  - περιέχει έναν πολύ καλό editor που σας βοηθά να γράψετε τον κώδικα σας δομημένα, ενώ ως μεταγλωττιστή (compiler) χρησιμοποιεί τον g++.
  - μπορείτε να το κατεβάσετε δωρεάν από τη σελίδα http://www.codeblocks.org/downloads/26

Στα πλαίσια του μαθήματος θα χρησιμοποιήσουμε ένα διαδυκτιακό IDE που βρίσκεται στη διεύθυνση https://www.tutorialspoint.com/compile_cpp_online.php

Η Γλώσσα Προγραμματισμού C++

Συμπερίληψη Βιβλιοθήκης Δυνατότητα χρήσης των εντολών που βρίσκονται στην βιβλιοθήκη (iostream, cmath, string)

Σχόλια (όπως και οτιδήποτε συμπεριλαμβάνεται μεταξύ /* και */). Δεν λαμβάνεται υπόψη από τον μεταγλωττιστή

```
//my first program
#include <iostream>
using namespace std;
int main(){                          // κύρια συνάρτηση προγράμματος
   cout << "C++ is easy!";           // εντολή εξόδου
return 0;
}                                    //τέλος προγράμματος
```

int main() η βασική συνάρτηση του προγράμματος. Όλες οι εντολές της συνάρτησης πρέπει να συμπεριληφθούν ανάμεσα σε άγκιστρα { }. Κάθε πρόγραμμα στη C++ αποτελείται από μία βασική συνάρτηση.

namespace Για αποφυγή συγκρούσεων μεταβλητών/συναρτήσεων με το ίδιο όνομα. Διαφορετικά θα έπρεπε να γράψουμε `std::cout << "C++ is easy!";`

Εντολή εξόδου- cout

Με την εντολή **cout** μπορούμε να εμφανίσουμε ένα μήνυμα στην οθόνη.

```
//multiple cout commands
#include <iostream>
using namespace std;
int main(){
 cout << "C++ is easy!";
 cout << "C++ is flexible!";
 cout << "C++ is my choice!";
return 0;
}
```

```
C++ is easy!C++ is flexible!C++ is my choice!
```

```
cout << "C++ is easy! C++ is flexible! C++ is my choice!";
cout << "C++ is easy! " << "C++ is flexible! " << "C++ is my choice!";
```

Εντολή εξόδου- Μορφοποίηση

Αλλαγή γραμμής **endl**

```
//multiple cout commands
#include <iostream>
using namespace std;
int main(){
  cout << "C++ is easy!" << endl;
  cout << "C++ is flexible!" << endl;
  cout << "C++ is my choice!" << endl;
return 0;
}
```

```
C++ is easy!
C++ is flexible!
C++ is my choice!
```

```
cout <<"C++ is easy!"<< endl <<"C++ is flexible!"<< endl <<"C++ is my choice!";
```

Εντολή εξόδου- Μορφοποίηση

Καθορισμός μεγέθους διαστήματος εκτύπωσης **setw**. Η εντολή αυτή βρίσκεται στη βιβλιοθήκη iomanip. Δέχεται ως παράμετρο ένα ακέραιο και καθορίζει το διάστημα της εκτύπωσης

```
#include<iostream>
#include<iomanip>      // περιλαμβάνει την εντολή setw
using namespace std;
int main(){
    cout<<"Keep"<<setw(5)<<"calm"<<endl;
    cout<<setw(6)<<"and"<<endl;
    cout<<"learn"<<setw(3)<<"to"<<setw(5)<<"code";
return 0;
}
```

```
Keep_calm
_ _ _and
learn_to_code
```

Εντολή εξόδου- Μορφοποίηση

Ακρίβεια δεκαδικών ψηφίων **setprecision**. Δέχεται ως παράμετρο έναν ακέραιο αριθμό, που καθορίζει τον αριθμό των δεκαδικών ψηφίων που θα τυπωθούν.
Αν θέλουμε να έχουμε ακριβώς τον αριθμό των ψηφίων που καθορίζει η εντολή, είναι απαραίτητο να γράψουμε την εντολή **fixed** πριν από την εντολή setprecision.
Τόσο η setprecision όσο και η fixed συμπεριλαμβάνονται στη βιβλιοθήκη iomanip.

```
#include<iostream>
#include<iomanip>
#include<cmath>        //για την εντολή sqrt
using namespace std;
int main(){
    cout<<fixed<<setprecision(3)<<sqrt(2)<<endl;     // 1.414
return 0;
}
```

Μεταβλητές και Σταθερές

Τα δεδομένα αποθηκεύονται με δύο τρόπους, είτε ως μεταβλητές είτε ως σταθερές.

- Μία μεταβλητή περιέχει μία τιμή, η οποία μπορεί να αλλάξει κατά την εκτέλεση του προγράμματος.
- Οι σταθερές έχουν σε όλη την διάρκεια της εκτέλεσης μία προκαθορισμένη τιμή που δεν αλλάζει.

Τύπος	Διάκριση	Bytes	Εμβέλεια
char	Χαρακτήρας	1	-128 έως 127
short	ακέραιος αριθμός	2	-32,768 έως 32,767
int	ακέραιος αριθμός	4	-2,147,483,648 έως 2,147,483,647
long long	ακέραιος αριθμός	8	$(-2^{63}+1)$ έως $(+2^{63}+1)$
float	πραγματικός αριθμός	4	1.2E-38 έως 3.4E+38
double	πραγματικός αριθμός	8	2.2E-308 έως 1.8E+308
long double	πραγματικός αριθμός	16	3.4E-4932 έως 1.2E+4932
bool	λογικές τιμές	1	true ή false

Μεταβλητές και Σταθερές

Για να χρησιμοποιηθεί, όμως, μία μεταβλητή πρέπει πρώτα να δηλωθεί, να της δοθεί δηλαδή ένα όνομα και να επιλεγεί ο τύπος δεδομένων που θα αποθηκεύει.

- Η δήλωση γίνεται γράφοντας έναν τύπο δεδομένων, ακολουθούμενο από ένα όνομα.

```
int num;              // δηλώνουμε μία μεταβλητή ακέραιου τύπου με όνομα num
double ratio;         // δηλώνουμε μία μεταβλητή δεκαδικού τύπου με όνομα ratio
bool check = false;         // δηλώνεται η check και αρχικοποιείται με τιμή false
char alpha='A', beta='B';  // δηλώνονται οι alpha και beta και αρχικοποιούνται με
                           // τιμές 'A' και 'B'
```

- Οι σταθερές δηλώνονται είτε όπως οι μεταβλητές, με προσθήκη της δεσμευμένης λέξης const στην αρχή, είτε με την οδηγία define. Κανένας από τους δύο τρόπους δεν επιτρέπει να αλλάξει η τιμή μετά την αρχικοποίηση.

```
#include<iostream>
using namespace std;
#define FPA_PERCENT 0.23          // Δήλωση σταθερής τιμής με define
int main(){
 const float fpa_percent = 0.23;  // Δήλωση σταθερής τιμής με const
return 0;
}
```

Εντολή εισόδου- cin

Η εντολή για εισαγωγή δεδομένων από το πληκτρολόγιο είναι η **cin** (standard input). Η εισαγωγή των δεδομένων για τον πιο κάτω κώδικα μπορεί να γίνει σε μία γραμμή, διαχωρίζοντας τα δεδομένα με κενό, ή εισάγοντας τα δεδομένα σε ξεχωριστές γραμμές:

```
int num1;
cin >> num1;            // εντολή εισόδου
```

```
int n;
double pi;
char let;
cin >> n >> pi >> let;
```

```
#include<iostream>
using namespace std;
int main(){
 int ak1, ak2, ak3;
 double de1, de2;
 bool test;
 cin >> ak1 >> de1;                    //input: 56 78.7
 cin >> ak2 >> de2;                    //input: 62 67.3
 cin >> ak3 >> test;                   //input: 64 false
 cout << ak1 << " " << ak2 << " " << ak3 << endl; //output: 56 62 64
 cout << de1 << " " << de2 << endl;    //output: 78.7 67.3
 cout << test << endl;                 //output: 0 (0-false, 1-true)
return 0;
}
```

Τελεστές

Τελεστής Ανάθεσης Τιμής (=)

- Απόδοση τιμής σε μία μεταβλητή

Αριθμητικοί τελεστές (+ , - , * , / , %)

Τελεστές προσαρμογής τύπου (type casting)

- μετατροπή δεδομένου ενός ενός τύπου σε δεδομένο διαφορετικού τύπου.

```
#include<iostream>
#include<iomanip>
using namespace std;
int main(){
    int a,b,c;
    float mo;
    cout<<"Input three integers: "<<endl;
    cin>>a>>b>>c;
    mo=(float)(a+b+c)/3;   //Μετατρέπουμε το άθροισμα σε δεκαδικό
    cout<<"MO="<<fixed<<setprecision(2)<<mo;
return 0;
}
```

Προτεραιότητα τελεστών

( )	παρενθέσεις
*, /, %	πολλαπλασιασμός, διαίρεση, υπόλοιπο ακέραιας διαίρεσης
+, -	πρόσθεση, αφαίρεση

Παράδειγμα 1

Η απόσταση μεταξύ δύο σημείων δίνεται από τον τύπο

$$d = \sqrt{(X2 - X1)^2 + (Y2 - Y1)^2}$$

όπου X1, Y1 και X2, Y2 οι συντεταγμένες 2 σημείων. Να δημιουργήσετε πρόγραμμα, το οποίο να δέχεται τις συντεταγμένες των 2 σημείων και να υπολογίζει την απόσταση d. Το αποτέλεσμα να εμφανίζεται με ακρίβεια 2 δεκαδικών ψηφιών.

Σημείωση: Να γίνει χρήση των έτοιμων συναρτήσεων sqrt και pow της βιβλιοθήκης cmath.

Παράδειγμα εισόδου

```
1 1 0 0
```

Παράδειγμα εξόδου

```
1.41
```

Δομές Διακλάδωσης

Απλή δομή διακλάδωσης

```
if (λογική έκφραση) {
      εντολές που εκτελούνται, αν ισχύει η λογική έκφραση
}
else {
      εντολές που εκτελούνται, αν δεν ισχύει η λογική έκφραση
}
```

```
#include<iostream>
using namespace std;
int main(){
    int num;
    cout<<"Num=";
    cin>>num;
    if(num%2==0){

        cout<<"Artios";
    }
    else {
        cout<<"Perittos";
    }
return 0;
}
```

Δομές Διακλάδωσης

Συγκριτικοί τελεστές

Οι συγκριτικοί τελεστές χρησιμοποιούνται, για να σχηματίσουμε λογικές συνθήκες. Μία λογική συνθήκη απαρτίζεται από τρία μέρη:

- τον συγκριτικό τελεστή στη μέση
- τις εκφράσεις τις οποίες συγκρίνουμε στα δύο άκρα του.
- το αποτέλεσμα είναι μία λογική τιμή, είτε αληθής (true) είτε ψευδής (false).

==	ίσο
!=	διάφορο / άνισο
>	μεγαλύτερο
<	μικρότερο
>=	μεγαλύτερο ή ίσο
<=	μικρότερο ή ίσο

```
cout << (7 == 5);          // 0 (false)
cout << (5 > 4);           // 1 (true)
cout << (3 != 2);          // 1 (true)
cout << (6 >= 6);          // 1 (true)
cout << (5 < 5);           // 0 (false)
```

Παράδειγμα 2

Ένα σχολείο δημιουργεί τους τριψήφιους αριθμούς μητρώου των μαθητών του ως εξής:

- Αν το ψηφίο των μονάδων είναι άρτιος αριθμός, τότε σημαίνει ότι ο μαθητής είναι αγόρι, διαφορετικά είναι κορίτσι.
- Αν το ψηφίο των δεκάδων είναι το 5, τότε σημαίνει ότι ο μαθητής είναι εκτοπισμένος, διαφορετικά δεν είναι.
- Αν ο αριθμός των εκατοντάδων είναι μεγαλύτερος του 7, τότε σημαίνει ότι ο μαθητής είναι από πολύτεκνη οικογένεια, διαφορετικά δεν είναι.

Να δημιουργήσετε πρόγραμμα το οποίο να δέχεται τον αριθμό μητρώου (ΑΜ) του μαθητή και να τυπώνει τις πληροφορίες του.

Δομές Διακλάδωσης

Λογικοί Τελεστές

Στα παραδείγματα που παρουσιάσαμε μέχρι στιγμής, οι λογικές εκφράσεις ήταν απλές και εξέταζαν μόνο μία συνθήκη.

- Πολλές φορές, όμως, είναι αναγκαίο να εξετάσουμε ταυτόχρονα περισσότερες από μία συνθήκες.
- Αυτό επιτυγχάνεται με τη χρήση των λογικών τελεστών.

Οι λογικοί τελεστές, οι οποίοι θα εξετάσουμε, είναι ο τελεστής And (&&), ο τελεστής Or (||) και ο τελεστής Not (!). Ο πίνακας αληθείας των λογικών τελεστών είναι ο ακόλουθος:

X	Y	X && Y	X || Y	!X
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Η προτεραιότητα των τελεστών είναι:

1. !(Not)
2. && (And)
3. || (Or)

Δομές Διακλάδωσης

Στις περιπτώσεις που πρέπει να συνδυάσουμε διάφορες λογικές εκφράσεις, για να αποφασίσουμε ποιες εντολές θα χρησιμοποιηθούν, τότε έχουμε **ένθετη δομή διακλάδωσης**.

- Μόλις βρεθεί λογική έκφραση που είναι αληθής (true), εκτελούνται οι αντίστοιχες εντολές και η δομή διακλάδωσης τερματίζεται.
- Είναι σαν να έχουμε πολλά if, το ένα μετά το άλλο και θα εκτελεστούν οι εντολές του πρώτου, του οποίου η λογική έκφραση θα είναι true. Οι υπόλοιπες εντολές θα αγνοηθούν.

```
if (λογική έκφραση 1) {
      εντολές που εκτελούνται, αν ισχύει η λογική έκφραση 1
}
else if (λογική έκφραση 2) {
      εντολές που εκτελούνται, αν ισχύει η λογική έκφραση 2
}
else {
      εντολές που εκτελούνται, αν δεν ισχύει η λογική έκφραση 1 ή 2
}
```

Παράδειγμα 3

Ένα τυπογραφείο χρεώνει κλιμακωτά τους πελάτες του ως εξής:

- Τα πρώτα 100 βιβλία [1-100] προς 8 ευρώ το ένα.
- Τα επόμενα 400 βιβλία [101-500] προς 6.40 ευρώ το ένα.
- Τα υπόλοιπα βιβλία (περισσότερα από 500) προς 3.50 ευρώ το ένα.

Να δημιουργήσετε πρόγραμμα το οποίο να δέχεται το πλήθος των βιβλίων και να υπολογίζει το κόστος.

Δομές Διακλάδωσης

Περιπτωσιακή Δομή – Η εντολή switch

Με την εντολή αυτή, μπορούμε να εκτελέσουμε μια ομάδα εντολών αναλόγως της τιμής που θα έχει η μεταβλητή ελέγχου.

Η διαφορά με το if είναι ότι δεν εξετάζει κάποια λογική έκφραση, αλλά τις τιμές που μπορεί να πάρει μια μεταβλητή ελέγχου

- Η μεταβλητή ελέγχου πρέπει να είναι διακριτού τύπου (integer, char)
- Όταν βρεθεί περίπτωση που θα έχει την ίδια τιμή με την μεταβλητή ελέγχου, τότε εκτελούνται οι εντολές που βρίσκονται από κάτω μέχρι να βρεθεί η εντολή break. Όταν εκτελεστεί η εντολή break, η switch τερματίζεται
- Οι εντολές που βρίσκονται μετά το default, θα εκτελεστούν εφόσον δε βρεθεί καμιά περίπτωση που να είναι ίση με την τιμή της μεταβλητής ελέγχου

```
switch(μεταβλητή ελέγχου){
    case περίπτωση 1:
        εντολές;
        break;
    case περίπτωση 2:
        εντολές;
        break;
    // Μπορείτε να έχετε όσες περιπτώσεις θέλετε
    default:
        εντολές;
}
```

Δομές Επανάληψης- Ο βρόχος for

Ο βρόχος for

- είναι η πιο απλή επαναληπτική δήλωση
- μας επιτρέπει να δημιουργούμε επαναληπτικές δομές με την ελάχιστη δυνατή χρήση κώδικα.
- χρησιμοποιείται όταν γνωρίζουμε από πριν τον ακριβή αριθμό των επαναλήψεων.

```
          (1)                (2)                (3)
for (αρχική τιμή μετρητή; συνθήκη ελέγχου μετρητή; μεταβολή τιμής μετρητή) {
      εντολές εντός του βρόχου
}
```

```
#include<iostream>
using namespace std;
int main(){
char letter;
  for (letter = 'a'; letter <= 'z'; letter++){
      cout << letter << endl;
   }
  for (double fx = 0.5; fx < 3.5; fx += 0.5)
      cout << fx << endl;
return 0;
}
```

Δομές Επανάληψης- Φωλιασμένες εντολές for

Σε πολλές περιπτώσεις μπορούμε να χρησιμοποιήσουμε επαναληπτικές εντολές μέσα σε άλλες επαναληπτικές εντολές.

- Στην περίπτωση αυτή, για κάθε επανάληψη της εξωτερικής εντολής, η εσωτερική θα εκτελέσει όλες τις δικές της επαναλήψεις.

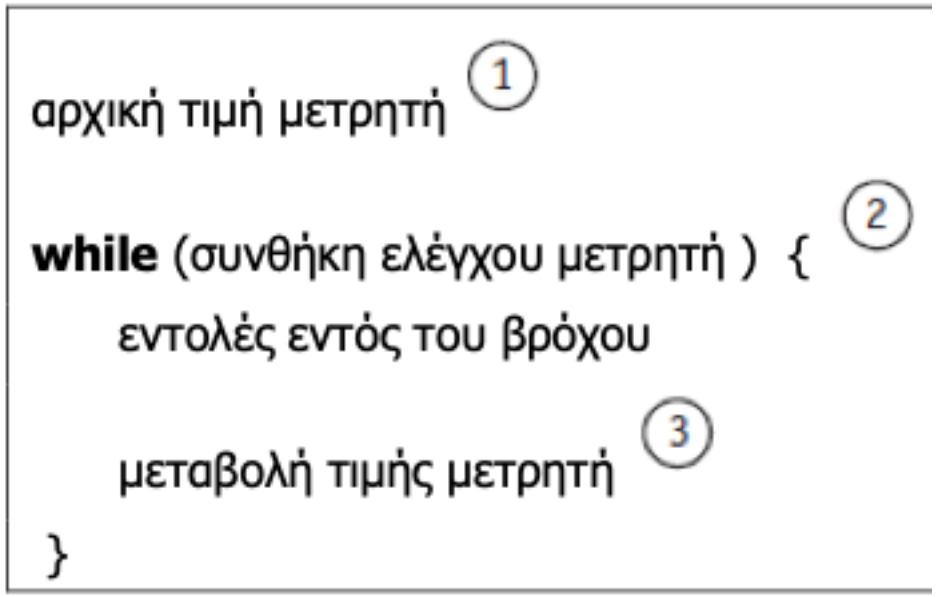
```
#include<iostream>
using namespace std;
int main(){

 for (int i = 1; i <= 8; i++){
    for (int j = 1; j <= i; j++){
          cout << "*";
 }
 cout << endl;
}
return 0;
}
```

Δομές Επανάληψης- Ο βρόχος while

Ο βρόχος while

- Για να επαναλάβει τις εντολές του ο βρόχος while, ελέγχει τη λογική συνθήκη και εάν αυτή είναι αληθής, τότε επαναλαμβάνει τις εντολές που του έχουμε ορίσει.
- Όταν εκτελέσει όλες τις εντολές του, ο έλεγχος μεταβιβάζεται πάλι στην αρχή και επανεξετάζει τη λογική συνθήκη.
- Όσο η λογική συνθήκη παραμένει αληθής, ο βρόχος θα επαναλαμβάνεται.
- Όταν η λογική συνθήκη γίνει ψευδής, τότε ο έλεγχος του προγράμματος θα συνεχίσει με το υπόλοιπο πρόγραμμα μετά από αυτόν.




```
int i = 1;              // αρχική τιμή μετρητή
while(i <= 10) {        // λογική συνθήκη ελέγχου μετρητή
  cout << i << " ";
  i++;                        // μεταβολή τιμής μετρητή
}
```

Δομές Επανάληψης- Ο βρόχος do

Ο βρόχος do

- Σε έναν βρόχο while η λογική συνθήκη ελέγχεται στην αρχή. Αν αυτή είναι ψευδής, τότε το σώμα του βρόχου δεν εκτελείται καθόλου.
- Σε μερικές περιπτώσεις, όμως, θέλουμε να εξασφαλίσουμε ότι οι εντολές του βρόχου θα εκτελεστούν έστω και μία φορά, ανεξάρτητα από την τιμή της λογικής συνθήκης.
- Σε αυτή την περίπτωση χρησιμοποιούμε τον βρόχο do, ο οποίος κάνει τον έλεγχο της λογικής συνθήκης στο τέλος του βρόχου.

```
#include<iostream>
using namespace std;
int main(){
int n;

do {
   cin >> n;
   cout << "You pressed: "<<n <<endl;
 }
 while (n != 0);

return 0;
}
```

Παράδειγμα 4

Ένας θετικός ακέραιος αριθμός ονομάζεται «Τέλειος», αν το άθροισμα των θετικών αριθμών που τον διαιρούν (διαιρέτες), πλην του εαυτού του, είναι ίσο με τον αριθμό. Για παράδειγμα, το 6 είναι τέλειος γιατί έχει ως διαιρέτες τους 1,2 και 3 και ισχύει 1+2+3=6.

Να δημιουργήσετε πρόγραμμα το οποίο να βρίσκει όλους τους τέλειους αριθμούς στο πεδίο [2..20000] και να τους εμφανίζει στην οθόνη.

Παράδειγμα 5

Έστω ότι έχουμε τον αλγόριθμο που βλέπουμε στα δεξιά μας.

Για παράδειγμα, αν n=22 τότε θα τυπώσει τους αριθμούς 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1 (16 επαναλήψεις).

Να δημιουργήσετε πρόγραμμα το οποίο να δέχεται ένα ζευγάρι ακεραίων αριθμών X,Y (0<X,Y<=100,000) και εφαρμόζοντας τον πιο πάνω αλγόριθμο σε όλους τους αριθμούς που βρίσκονται μέσα στο διάστημα X,Y (συμπεριλαμβανομένων), να τυπώνει το μέγιστο πλήθος επαναλήψεων, το οποίο μπορεί να προκύψει.

Παράδειγμα εισόδου

```
1 10
```

Παράδειγμα εξόδου

```
20
```

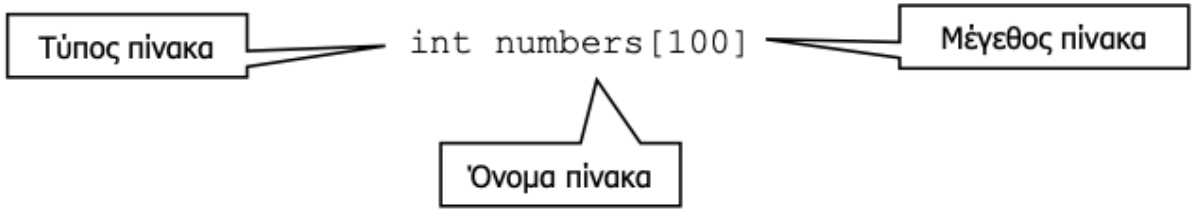
Επεξήγηση: Το μέγιστο πλήθος επαναλήψεων προκύπτει με τον αριθμό 9.

```
Διάβασε n
Τύπωσε n
Όσο το n ≠ 1 επανάλαβε {
   Αν n περιττός τότε{
      n=3n+1
    }
   Αλλιώς{
      n=n/2
    }
  Τύπωσε n
}
```

Πίνακες

Οι πίνακες μάς δίνουν τη δυνατότητα να ομαδοποιήσουμε δεδομένα του ιδίου τύπου.

- Αναφορά στον πίνακα γίνεται μέσω ενός ονόματος (αναγνωριστικού
- για πρόσβαση στα δεδομένα ενός πίνακα χρησιμοποιείται το όνομα και ένας αριθμός (δείκτης - index), ο οποίος καθορίζει τη θέση του στοιχείου μέσα στον πίνακα.



Ένας πίνακας θα πρέπει να δηλωθεί πριν χρησιμοποιηθει μέσα σε ένα πρόγραμμα. Για τον ορισμό ενός πίνακα παρέχουμε τρία στοιχεία.

- Το όνομα του πίνακα, τον τύπο δεδομένων των στοιχείων του, το πλήθος των θέσεων του πίνακα.

```
int grades[4] = {16, 18, 15, 20};
```

```
int primes[] = {1,2,3,5,7};    //Δεν χρειάζεται να δηλώνουμε το μέγεθος.
char characters[3] = {'a', 'b', 'c', 'd'};  //Σφάλμα! Περισσότερα στοιχεία.
int a[5] = {12,5,4};                        // a → {12, 5, 4, 0, 0}
```

Πίνακες- Εισαγωγή και εμφάνιση στοιχείων του πίνακα

```
char arr[10];
// Εισαγωγή στοιχείων σε πίνακα
for (int i=0; i<10; i++)
cin >> arr[i];
// Εμφάνιση στοιχείων του πίνακα
for (int i=0; i<10; i++)
    cout << arr[i] << " ";
```

Συμβολοσειρές (Strings)

Στη C++ δεν υπάρχει τύπος δεδομένων για τη δήλωση μίας συμβολοσειράς, όπως υπάρχει για ακέραιους αριθμούς, χαρακτήρες κ.λπ.

Υπάρχει, όμως, η δυνατότητα να δηλώσουμε μία συμβολοσειρά ως αντικείμενο, χρησιμοποιώντας τη βιβλιοθήκη <string>.

- Επειδή οι συμβολοσειρές είναι αντικείμενα και όχι τύποι δεδομένων, συνεπάγεται ότι θα έχουν ιδιότητες και έτοιμες συναρτήσεις.
- Μετά τη δήλωση ενός αντικειμένου τύπου string, μπορούμε να το χειριστούμε ως πίνακα χαρακτήρων.

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string a, b;
    cin >> a >> b;
    if(a[0]==b[0])
        cout << "1";
    else
        cout << "0";
return 0;
}
```

Συμβολοσειρές (Strings)

Συναρτήσεις της βιβλιοθήκης string

- **size()** επιστρέφει το μέγεθος της συμβολοσειράς,
- **clear()** διαγράφει το περιεχόμενο μίας συμβολοσειράς,
- **empty()** ελέγχει αν η συμβολοσειρά είναι άδεια
- **getline()** διαβάζει ολόκληρη τη γραμμή.

```
string st1;
cin >> st1;                          // Input: Hello world
cout << st1 << endl;                 // Output: Hello
cout << st1.size() << endl;          // 5
string st1;
getline(cin, st1);                   // Input: Hello world
cout << st1 << endl;                 // Output: Hello world
cout << st1.size() << endl;          // 11
```

Συμβολοσειρές (Strings)

Πίνακες συμβολοσειρών

Μπορούμε να δηλώσουμε πίνακες με συμβολοσειρές με τον ίδιο τρόπο που θα δηλώναμε έναν πίνακα ακεραίων.

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string A[5], st;
    for(int i=0; i<5; i++)
        cin >> A[i];
    for(int i=0; i<5; i++)
        st += A[i][0];   // Προσθήκη 1ου χαρακτήρα στη συμβολοσειρά
    cout<<st;
return 0;
}
```

Παράδειγμα 6

Η Αριάδνη έχει κρύψει τον κωδικό της μέσα σε μία συμβολοσειρά. Δεν είναι σίγουρη, όμως, αν το έκανε σωστά. Να δημιουργήσετε πρόγραμμα, το οποίο να δέχεται τον κωδικό και τη συμβολοσειρά και να τυπώνει τις θέσεις των χαρακτήρων του κωδικού που εμφανίζονται μέσα στη συμβολοσειρά. Αν ο κωδικός δεν υπάρχει μέσα στη συμβολοσειρά, να τυπώνεται ο αριθμός -1. Οι χαρακτήρες του κωδικού πρέπει να εμφανίζονται μέσα στη συμβολοσειρά με τη σωστή σειρά, αλλά όχι κατά ανάγκη ο ένας δίπλα από τον άλλο. Τόσο ο κωδικός όσο και η συμβολοσειρά αποτελούνται από κεφαλαία γράμματα του λατινικού αλφαβήτου.

Παράδειγμα εισόδου

```
STOP
ABDSTKLOTP
```

Παράδειγμα εξόδου

```
3 4 7 9
```

Αρχεία (Files)

Μέχρι στιγμής, η είσοδος δεδομένων στα προγράμματά μας γινόταν από το πληκτρολόγιο και η έξοδος πληροφοριών γινόταν στην οθόνη.

- Η χρήση των εντολών cin/cout δεν είναι, όμως, πολύ βολική, όταν έχουμε να διαχειριστούμε μεγάλο όγκο δεδομένων.
- Σε τέτοιες περιπτώσεις ενδείκνυται η χρήση αρχείων.
- Μπορούμε να χρησιμοποιήσουμε τα αρχεία, για να διαβάσουμε δεδομένα και να γράψουμε σε αυτά πληροφορίες.
- Η βιβλιοθήκη που περιέχει τις εντολές που αφορούν στα αρχεία είναι η <fstream>.
- Η εντολή που χρησιμοποιείται για την ανάγνωση ενός αρχείου είναι η ifstream. Με την εντολή ifstream δημιουργούμε μία ροή (stream) ανάγνωσης προς ένα εξωτερικό αρχείο.
- Αντίστοιχα, με την εντολή ofstream δημιουργούμε μία ροή εγγραφής προς ένα εξωτερικό αρχείο.
- Το αρχείο εισόδου πρέπει να βρίσκεται στον ίδιο φάκελο με το πρόγραμμα,
- Το αρχείο εξόδου δεν είναι απαραίτητο να προϋπάρχει, αφού θα δημιουργηθεί από το πρόγραμμα στον ίδιο φάκελο.

Αρχεία (Files)

```
#include<fstream>
using namespace std;

int main(){
    ifstream infile("in.txt");          // Ροή ανάγνωσης - infile
    ofstream outfile("out.txt");        // Ροή εγγραφής - outfile
    int i, A[100], sum = 0;
    float mo;
    for(i=0; i<100; i++){
        infile >> A[i];
        sum += A[i];                    // Άθροισμα στοιχείων
    }
    mo = sum / 100.0;
    for(i=0; i<100; i++)
        if(A[i]>mo)
            outfile << A[i] << endl;    // Εμφάνιση στοιχείων
    infile.close();
    outfile.close();
return 0;
}
```

Αρχεία (Files)

Εγγραφή στο τέλος ενός αρχείου

```
ofstream myout("file.txt", (ios::out | ios::app));
```

Έλεγχος για το τέλος ενός αρχείου

Πολλές φορές δεν γνωρίζουμε το πλήθος των στοιχείων ενός αρχείου. Για να το υπολογίσουμε, μπορούμε να διαβάσουμε όλα τα στοιχεία του αρχείου μέχρι να φτάσουμε στο τέλος του. Αυτό μπορεί να γίνει με τη χρήση της συνάρτησης **eof()**, η οποία επιστρέφει την τιμή true όταν φτάσουμε στο τέλος του αρχείου.

```
ifstream fin("persons.txt");
while(!fin.eof()){              // Έλεγχος για το τέλος του αρχείου
    fin >> names[i] >> ages[i];
    sum += ages[i];
    i++;                        // Αύξηση πλήθους ατόμων
}
```

Παράδειγμα 7

Να δημιουργήσετε πρόγραμμα, το οποίο να διαβάζει δύο αρχεία (file1.txt, file2.txt), να τα συγκρίνει και να εμφανίζει στην οθόνη τα μηνύματα «Files are the same», αν τα δύο αρχεία είναι τα ίδια, ή «Files are the not the same», στην αντίθετη περίπτωση.

Συναρτήσεις (Functions)

Τμηματικός προγραμματισμός (Modular programming) είναι μία τεχνική ανάπτυξης προγραμμάτων, της οποίας κύριο χαρακτηριστικό είναι η διάσπαση ενός προβλήματος σε πιο απλά τμήματα προγραμμάτων, έτσι ώστε να διευκολυνθεί η αποσφαλμάτωση και να μειωθεί η έκταση του κώδικά. Αυτά τα τμήματα κώδικά καλούνται υποπρογράμματα.

- Υποπρόγραμμα θεωρείται ένα αυτόνομο κομμάτι κώδικα, το οποίο εκτελεί ένα συγκεκριμένο έργο και ορίζεται ξεχωριστά από την κύρια συνάρτηση (main).
- Η σωστή εφαρμογή του τμηματικού προγραμματισμού διασφαλίζει τη σωστή και απλούστερη δημιουργία προγραμμάτων. Επίσης, διευκολύνει την κατανόηση του τρόπου λειτουργίας ενός προγράμματος.

Συναρτήσεις (Functions)

Ορισμός και κλήση συναρτήσεων

Η συνάρτηση (function) είναι ένας τύπος υποπρογράμματος που υπολογίζει και επιστρέφει μόνο μία τιμή στην κύρια συνάρτηση (main).

```
#include <iostream>
using namespace std;

int add_2_nums(int a, int b){          // Ορισμός συνάρτησης
      return a + b;                    // Επιστροφή αθροίσματος
}                                      // Τέλος σώματος συνάρτησης

int main() {                           // Κύρια συνάρτηση (main)
      cout << add_2_nums(65,97);       // Κλήση συνάρτησης add_2_nums
return 0;                              // μέσα στην κύρια συνάρτηση
}                                      // Τέλος κύριας συνάρτησης
```

Συναρτήσεις (Functions)

Για κάθε συνάρτηση πρέπει να ορίζουμε τα εξής:

- **Τύπος επιστρεφόμενης τιμής**: Η συνάρτηση μπορεί να επιστρέψει όλους τους βασικούς τύπους που γνωρίζουμε (int, double, char, boolean), ακόμα και συμβολοσειρές (string). Αν η συνάρτηση δεν θα επιστρέφει κάποια τιμή, ορίζεται με τύπο void. Αν επιστρέφει κάποια τιμή, αυτή θα εμφανίζεται μετά τη λέξη return.
- **Όνομα συνάρτησης**: Ισχύουν οι ίδιοι κανόνες ονοματολογίας με τις μεταβλητές.
- **Παράμετροι**: Μεταβλητές που επιτρέπουν το πέρασμα της τιμής τους από ένα τμήμα προγράμματος σε άλλο. Μία συνάρτηση μπορεί να έχει μηδέν ή περισσότερες παραμέτρους.

```
int add_2_nums(int a, int b){
```

Τοπικές (local) και καθολικές (global) μεταβλητές

Συναρτήσεις (Functions)

Παράμετροι τιμών (by value) και παράμετροι αναφοράς (by reference) Οι συναρτήσεις με τη χρήση της εντολής return μπορούν να επιστρέψουν μόνο μία τιμή. Σε περίπτωση που θέλουμε η συνάρτησή μας να «επηρεάσει» περισσότερες από μία μεταβλητές, χρησιμοποιούμε την κλήση παραμέτρων με αναφορά. Με αυτό τον τρόπο περνάμε στη συνάρτηση τη διεύθυνση της μεταβλητής, με τη χρήση του τελεστή & και όχι την τιμή της.

```
// Συνάρτηση τύπου void. Δεν επιστρέφει τιμή – δεν υπάρχει return
void byvalue(int x){                  // Παράμετρος τιμής
      x = x + 1;                      // Αλλαγή τιμής παραμέτρου
      cout << x << endl;              // Εντολή εξόδου
}
// Συνάρτηση τύπου void. Δεν επιστρέφει τιμή – δεν υπάρχει return
void byreference(int &y){   // Παράμετρος αναφοράς με τη χρήση του &
      y = y + 1;                      // Η τιμή της μεταβλητής θα αλλάξει
      cout << y << endl;              // και μέσα στην κύρια συνάρτηση
}

int main() {                          // Κύρια συνάρτηση (main)
  int num = 5;
  byvalue(num);                                   // 6
  cout << "number=" << num << endl;               // number=5
  byreference(num);                               // 6
  cout << "number=" << num << endl;               // number=6
return 0;
}
```

Συναρτήσεις (Functions)

Πίνακες και συναρτήσεις

Ένας πίνακας αποτελείται από περισσότερες από μία τιμές,

- Για να περάσουμε έναν πίνακα σε μία συνάρτηση ως παράμετρο, το πρότυπο της συνάρτησης θα περιλαμβάνει τον πίνακα ως παράμετρο αναφοράς (δηλαδή θα περάσουμε τη διεύθυνση του πίνακα).
- Στις παραμέτρους αναφοράς με πίνακες δεν χρησιμοποιείται ο τελεστής &.
- Το μέγεθος του πίνακα δεν ορίζεται παραμετρικά στο πρότυπο της συνάρτησης, ενώ στην κλήση της συνάρτησης δηλώνουμε μόνο το όνομα του πίνακα.

```
#include<iostream>
#define N 20                    // Σταθερά για το μέγεθος του πίνακα
using namespace std;

void fill(int arr[]);           // Πρότυπο συνάρτησης fill
void print(int arr[]);          // Πρότυπο συνάρτησης print

int main(){                     // Κύρια συνάρτηση (main)
   int A[N];                    // Δήλωση πίνακα ακεραίων 20 θέσεων
   fill(A);                     // Καταχώριση στοιχείων
   print(A);                    // Εμφάνιση στοιχείων
return 0;
}                               // Τέλος κύριας συνάρτησης
void fill(int arr[]){           // Ορισμός συνάρτησης fill
   for(int i=0; i<N; i++)
       cin >> arr[i];
}                               // Τέλος συνάρτησης fill
void print(int arr[]){          // Ορισμός συνάρτησης print
   for(int i=0; i<N; i++)
       cout << arr[i] << " ";
}                               // Τέλος συνάρτησης print
```

Συναρτήσεις (Functions)

Αρχεία και συναρτήσεις

Αν θέλουμε να χρησιμοποιήσουμε αρχεία σε συνδυασμό με συναρτήσεις, τότε οι εντολές που χρησιμοποιούμε για δημιουργία ροής ανάγνωσης (ifstream) και ροής εγγραφής (ofstream) μπορούν να δηλωθούν εκτός της κύριας συνάρτησης. Με αυτό τον τρόπο, οι εντολές αυτές θα είναι προσβάσιμες από όσες συναρτήσεις χρησιμοποιήσουμε στο πρόγραμμα.

```
#include<fstream>                    // Για τη χρήση αρχείων
#include<cmath>                      // Για τη χρήση της sqrt
using namespace std;
// Οι ροές δηλώνονται εκτός της κύριας συνάρτησης main
ifstream fin("numbers.txt");     // Δημιουργία ροής ανάγνωσης
ofstream fout("squares.txt");    // Δημιουργία ροής εγγραφής
int cnt = 0;     // Καθολική μεταβλητή για το πλήθος των ακεραίων

void fill(int nums[]){      // Συνάρτηση καταχώρισης ακεραίων
    while(!fin.eof()){      // στον πίνακα, από το αρχείο numbers.txt
       fin >> nums[cnt];    // Διαβάζει και καταχωρίζει έναν ακέραιο
       cnt++;               // Αύξηση του μετρητή
    }
}
bool square(int n){   // Συνάρτηση για εντοπισμό τέλειου τετραγώνου
   int x = (int)(sqrt(n) + 0.5);
   if (x*x == n)
       return true;   // Επιστρέφει true αν έχουμε τέλειο τετράγωνο
return false;
}
```

Παράδειγμα 8

Να δημιουργήσετε πρόγραμμα, το οποίο να χρησιμοποιεί τη συνάρτηση rect, η οποία να δέχεται παραμετρικά το μήκος και το πλάτος ενός ορθογωνίου, να υπολογίζει και να επιστρέφει, ως παραμέτρους αναφοράς, το εμβαδόν και την περίμετρο του ορθογωνίου.

Πρότυπο συνάρτησης: **`void rect(double w,double l,double &e,double &p);`**

Παράδειγμα 9

Να δημιουργήσετε πρόγραμμα, το οποίο να καλεί μία συνάρτηση με το όνομα shift_right, η οποία να δέχεται παραμετρικά έναν πίνακα ακεραίων έξι θέσεων και έναν ακέραιο αριθμό N (0<=N<=6). Η συνάρτηση να μετακινεί κάθε στοιχείο του πίνακα Ν θέσεις δεξιά. Η καταχώριση των στοιχείων για τον πίνακα θα γίνεται στην κύρια συνάρτηση (main).

Παράδειγμα εισόδου

```
15 26 32 44 51 68
3
```

Παράδειγμα εξόδου

```
44 51 68 15 26 32
```

Δισδιάστατοι πίνακες (2D arrays)

Με τους **δισδιάστατους πίνακες (2D arrays)** έχουμε τη δυνατότητα να αποθηκεύσουμε στοιχεία ίδιου τύπου σε ένα ορθογώνιο πλέγμα κελιών, το οποίο αποτελείται από πολλές γραμμές και πολλές στήλες.

- Κάθε στοιχείο του πίνακα χαρακτηρίζεται από δύο μεταβλητές-δείκτες.
- Έναν δείκτη που υποδεικνύει τη γραμμή (στο πρώτο ζεύγος αγκυλών) και έναν που υποδεικνύει τη στήλη που βρίσκεται το στοιχείο (στο δεύτερο ζεύγος αγκυλών).
- Η αρίθμηση και των δύο δεικτών ξεκινά από το μηδέν.

```
arr[0][0] = 17;
arr[1][2] = 5;
arr[4][3] = arr[1][2] + 19;
```

```
int arr[5][3];
```

	στήλη j=0	στήλη j=1	στήλη j=2
γραμμή i=0	arr[0][0]	arr[0][1]	arr[0][2]
γραμμή i=1	arr[1][0]	arr[1][1]	arr[1][2]
γραμμή i=2	arr[2][0]	arr[2][1]	arr[2][2]
γραμμή i=3	arr[3][0]	arr[3][1]	arr[3][2]
γραμμή i=4	arr[4][0]	arr[4][1]	arr[4][2]

```
int arr[3][3] = { {1,2,3},{4,5,6},{7,8,9} };
```

```
int arr[4][4] = { {1,2,3},{4,5,6},{7,8,9} };
```

Δισδιάστατοι πίνακες (2D arrays)

Εισαγωγή στοιχείων σε δισδιάστατο πίνακα πίνακες (2D arrays)

Για την εισαγωγή στοιχείων σε πίνακα μπορούμε να χρησιμοποιήσουμε δύο εμφωλευμένες δομές επανάληψης for, μαζί με δύο μεταβλητές-δείκτες.

- Μπορεί να γίνει χρήση συνάρτησης και ένας δισδιάστατος πίνακας να δοθεί ως παράμετρος αναφοράς στη συνάρτηση, τον οποίο η συνάρτηση θα γεμίζει.
- Οι δισδιάστατοι πίνακες, όπως και οι μονοδιάστατοι, όταν περνούν παραμετρικά σε μία συνάρτηση, δίδονται πάντοτε ως παράμετροι αναφοράς και δεν χρειάζεται να γίνει χρήση του τελεστή &.
- Η μόνη διάσταση που απαιτείται είναι αυτή των στηλών, η οποία πρέπει να είναι σταθερά, επιτρέπεται όμως να δοθούν και οι δύο διαστάσεις παραμετρικά.

Εμφάνιση στοιχείων σε δισδιάστατο πίνακα πίνακες (2D arrays)

Για την εμφάνιση των στοιχείων ενός δισδιάστατου πίνακα στην οθόνη χρησιμοποιούμε παρόμοιες εμφωλευμένες δομές επανάληψης με την εισαγωγή στοιχείων, αυτή τη φορά με χρήση εντολής εξόδου.

- Μετά από κάθε εσωτερικό βρόχο προχωρούμε στην επόμενη γραμμή, ώστε να ξεχωρίσουμε τα στοιχεία των γραμμών.
- Επιπρόσθετα, μπορεί να γίνει χρήση της συνάρτησης setw, ώστε να εμφανίζεται ομοιόμορφη, δεξιά στοίχιση των στοιχείων στην οθόνη.

Δισδιάστατοι πίνακες (2D arrays)

```
#include <iostream>
using namespace std;
#define ROWS 3          // Δηλώνουμε το πλήθος των γραμμών ως σταθερά
#define COLS 4          // Δηλώνουμε το πλήθος των στηλών ως σταθερά

void fill_array(int arr[][COLS]){    // Ο πίνακας περνά ως παράμετρος
    for (int i=0; i<ROWS; i++)       // αναφοράς δηλώνοντας μόνο το
        for (int j=0; j<COLS; j++) // πλήθος των στηλών
            cin >> arr[i][j];
}

int main(){
     int my_array[ROWS][COLS];          // Δήλωση πίνακα
     fill_array(my_array);              // Κλήση συνάρτησης εισόδου
return 0;
}
```

Παράδειγμα 10

Να δημιουργήσετε πρόγραμμα το οποίο να ελέγχει αν ένας τετραγωνικός πίνακας διαστάσεων 3x3 είναι έγκυρος πίνακας για το παιχνίδι SUDOKU. Για να είναι έγκυρος ένας πίνακας, πρέπει:

- να περιέχει μόνο τους αριθμούς από το 1 μέχρι το 9 και
- ο κάθε αριθμός να εμφανίζεται μόνο μία φορά

Το πρόγραμμα να διαβάζει έναν πίνακα SUDOKU, διαστάσεων 3x3 και να ενημερώνει τον χρήστη αν ο πίνακας είναι έγκυρος (Valid) ή όχι (Not valid).

Εγγραφές (Structures)

Εγγραφές ονομάζουμε συλλογές στοιχείων, οι οποίες μας δίνουν τη δυνατότητα να ομαδοποιήσουμε στοιχεία διαφορετικών τύπων.

Μία εγγραφή ορίζεται ως εξής:

a. Ξεκινά με τη δεσμευμένη λέξη struct και ακολουθεί το αναγνωριστικό της εγγραφής.
b. Τα άγκιστρα οριοθετούν τα μέλη της εγγραφής.
c. Το ερωτηματικό στο τέλος ορίζει το τέλος της εγγραφής.

```
struct <όνομα εγγραφής> {
 <τύπος 1ου μέλους> <όνομα 1ου μέλους>;
 <τύπος 2ου μέλους> <όνομα 2ου μέλους>;
 ...
 <τύπος N-οστού μέλους> <όνομα N-οστού μέλους>;
};
```

Εγγραφές (Structures)

Για παράδειγμα, εάν θέλουμε να αποθηκεύσουμε πληροφορίες για τα προϊόντα μίας υπεραγοράς. Για κάθε προϊόν έχουμε τις εξής πληροφορίες: κωδικός προϊόντος (ακέραιος), ποσότητα (ακέραιος), περιγραφή (συμβολοσειρά) και τιμή (πραγματικός). Η εγγραφή θα δηλωθεί όπως πιο κάτω, με το όνομα product:

```
struct product {            // Ξεκινούμε με τη δεσμευμένη λέξη struct
  int code, quantity;       // Πρώτο/Δεύτερο μέλος: Ακέραιοι
  string description;       // Τρίτο μέλος: Συμβολοσειρά
  double price;             // Τέταρτο μέλος: Πραγματικός αριθμός
};                          // Τέλος εγγραφής
```

Για να δηλώσουμε μεταβλητές τύπου εγγραφής, μπορούμε να κάνουμε τα εξής:

1. product apple;
2. product apple, banana, orange;
3. Για δήλωση εγγραφής και μεταβλητών ταυτόχρονα:

```
struct product {
  int code, quantity;
  string description;
  double price;
} apple, banana, orange;
```

Εγγραφές (Structures)

Για αναφορά στα μέλη μίας εγγραφής χρησιμοποιούμε την πιο κάτω γενική εντολή, όπου μετά το όνομα της εγγραφής ακολουθεί μία τελεία (.) και το όνομα του μέλους της εγγραφής στο οποίο αναφερόμαστε:

```
<όνομα μεταβλητής εγγραφής>.<όνομα μέλους εγγραφής>
```

Παραδείγματα:

```
cin >> apple.code;
orange.quantity = 60;
```

Παράδειγμα Προγράμματος Εγγραφών

Να δημιουργήσετε πρόγραμμα, το οποίο να ορίζει μία εγγραφή με το όνομα student, η οποία να περιέχει τα ακόλουθα μέλη: Αριθμός μητρώου (integer), Επώνυμο (string), Τηλέφωνο (integer) και Εκτοπισμένος (boolean)

Στη συνέχεια, να ορίσετε έναν πίνακα εγγραφών 20 θέσεων τύπου student, με το όνομα Β3. Το πρόγραμμα να δέχεται τα στοιχεία για τους 20 μαθητές από το πληκτρολόγιο.

Εγγραφές (Structures)

```
#include <iostream>
#include <string>
using namespace std;

struct student {              // Ορισμός εγγραφής student
      int am;                 // Μέλη εγγραφής
      string name;
      int tel;
      bool ekt;
};                            // Τέλος εγγραφής

int main(){                   // Κύρια συνάρτηση (main)
      student B3[20];         // Δήλωση πίνακα εγγραφών 20 θέσεων
      for (int i=0; i<20; i++)
            cin >> B3[i].am >> B3[i].name >> B3[i].tel >> B3[i].ekt;
return 0;
}
```

Παράδειγμα 11

Σας δίνεται το αρχείο players.txt με τις εξής πληροφορίες: κωδικός, επίθετο, ημερομηνία γέννησης και ποσοστό ευστοχίας. Πιο κάτω βλέπετε δείγμα του αρχείου:

```
101     James     11/09/1985     53.17
102     Scott     12/10/1983     17.14
103     Sherman   28/11/1979     21.89
```

Να δημιουργήσετε πρόγραμμα, το οποίο να:

1. δηλώνει μία εγγραφή με το όνομα players, η οποία θα περιέχει τα κατάλληλα μέλη για να αποθηκεύσει τις πληροφορίες που εμφανίζονται στο αρχείο
2. διαβάζει όλα τα στοιχεία από το αρχείο players.txt και να τα αποθηκεύει σε πίνακα εγγραφών. Να θεωρήσετε ότι ο μέγιστος αριθμός παικτών που μπορεί να περιέχει το αρχείο είναι 100
3. εμφανίζει στην οθόνη τα ονόματα των παικτών με ποσοστό ευστοχίας πάνω από 50.
4. Εμφανίζει στην οθόνη τα ονόματα των παικτών με ποσοστό ευστοχίας πάνω από 50 ταξινομημένα σε φθίνουσα σειρά ευστοχίας (δηλαδή στην $1^{η}$ γραμμή θα εμφανίζεται ο πιο εύστοχος παίχτης)